



SR UNIVERSITY

Campus Warangal

Program: II - B. Tech (CS& AI) Batch: 16

Name: B.Hrithin Reddy
No:2503A52471

Reg.

Department: Computer Science and AI Semester: I

AI Assisted Coding-Assignment Date: 22/09/2026

1. Question-1

Q1 – AI-Assisted Bubble Sort Analysis

Problem / Starter Code:

```
numbers = [64, 34, 25, 12, 22, 11, 90]
```

```
# TODO:
```

```
# Implement Bubble Sort in ascending order.
```

```
# Count comparisons and swaps.
```

```
# Display the sorted list.
```

Student Tasks:

- Implement Bubble Sort.
- Use AI to explain one complete pass of Bubble Sort.
- Count comparisons and swaps.
- Test at least three input lists.
- Ask AI about an early-exit optimisation.
- Compare the original and improved versions.

2. Prompt:

“Explain Bubble Sort step by step for the given list. Identify opportunities to reduce unnecessary comparisons. Do not write the complete program yet.”

Code : numbers = [64, 34, 25, 12, 22, 11, 90]

3.Code

```
numbers = [64, 34, 25, 12, 22, 11, 90]

comparisons = 0
swaps = 0

for pass_number in range(len(numbers) - 1):
    swapped = False

    # The last pass_number elements are already in their final positions.
    for index in range(len(numbers) - 1 - pass_number):
        comparisons += 1

        if numbers[index] > numbers[index + 1]:
            numbers[index], numbers[index + 1] = (
                numbers[index + 1], numbers[index]
            )
            swaps += 1
            swapped = True

    if not swapped:
        break

print("Sorted list:", numbers)
print("Comparisons:", comparisons)
print("Swaps:", swaps)
```

4.Output

```
Sorted list: [11, 12, 22, 25, 34, 64, 90]
Comparisons: 21
Swaps: 14
```

5.Observation

Bubble Sort successfully sorts the given list in ascending order by repeatedly comparing and swapping adjacent elements. The early-exit optimization reduces unnecessary comparisons when the list becomes sorted.

#-----

Question-2

Q2 – Linear Search vs Binary Search

2.Prompt

“Compare Linear Search and Binary Search for the given sorted list [5, 12, 18, 23, 31, 45, 56, 72, 89]. Explain why Binary Search requires sorted data, count the comparisons for both methods, and test first, middle, last, and absent elements.

Suggest one valid optimization."

Code: earch_numbers = [5, 12, 18, 23, 31, 45, 56, 72, 89]

search_target = 45

3.Code

```
numbers = [5, 12, 18, 23, 31, 45, 56, 72, 89]
def linear_search(numbers, target):
    """Return the index and comparison count for linear search."""
    comparisons = 0

    for index, number in enumerate(numbers):
        comparisons += 1

        if number == target:
            return index, comparisons
        # Optimization: because the list is sorted, no later value can match.
        if number > target:
            break
    return -1, comparisons
def binary_search(numbers, target):
    """Return the index and comparison count for binary search."""
    comparisons = 0
    left = 0
    right = len(numbers) - 1

    while left <= right:
        middle = (left + right) // 2
        comparisons += 1

        if numbers[middle] == target:
            return middle, comparisons
        elif numbers[middle] < target:
            left = middle + 1
        else:
            right = middle - 1

    return -1, comparisons

test_values = [
    ("first element", numbers[0]),
    ("middle element", numbers[len(numbers) // 2]),
```

```

        ("middle element", numbers[len(numbers) // 2]),
        ("last element", numbers[-1]),
        ("absent element", 50),
    ]
    print("Numbers:", numbers)
    print()
    for description, target in test_values:
        linear_index, linear_comparisons = linear_search(numbers, target)
        binary_index, binary_comparisons = binary_search(numbers, target)
        print(description.title(), "(target:", target, ")")
        print("  Linear search:", end=" ")
        if linear_index == -1:
            print("not found, comparisons =", linear_comparisons)
        else:
            print("found at index", linear_index, ", comparisons =", linear_comparisons)

        print("  Binary search:", end=" ")
        if binary_index == -1:
            print("not found, comparisons =", binary_comparisons)
        else:
            print("found at index", binary_index, ", comparisons =", binary_comparisons)
        print()

```

4. Output

```

Numbers: [5, 12, 18, 23, 31, 45, 56, 72, 89]

First Element (target: 5 )
  Linear search: found at index 0 , comparisons = 1
  Binary search: found at index 0 , comparisons = 3

Middle Element (target: 31 )
  Linear search: found at index 4 , comparisons = 5
  Binary search: found at index 4 , comparisons = 1

Last Element (target: 89 )
  Linear search: found at index 8 , comparisons = 9
  Binary search: found at index 8 , comparisons = 4

Absent Element (target: 50 )
  Linear search: not found, comparisons = 7
  Binary search: not found, comparisons = 3

```

5. Observation

Linear Search checks elements sequentially, while Binary Search repeatedly divides the sorted list into halves.

Binary Search generally requires fewer comparisons, especially for larger sorted datasets.

The choice depends on whether the data is sorted and how frequently searching is performed.

#-----

Question-3

3 question: Compare Sorting Algorithms

2. Prompt:

"Compare Bubble Sort, Selection Sort, and Insertion Sort using the given list [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]. Count their comparisons and swaps/shifts, explain their strengths and weaknesses, and identify which algorithm is most

suitable for nearly sorted data. Compare the results with their theoretical time complexities.”

3.code

```
data = [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]
def bubble_sort(values):
    numbers = values.copy()
    comparisons = 0
    swaps = 0

    for pass_number in range(len(numbers) - 1):
        swapped = False

        for index in range(len(numbers) - 1 - pass_number):
            comparisons += 1

            if numbers[index] > numbers[index + 1]:
                numbers[index], numbers[index + 1] = (
                    numbers[index + 1], numbers[index]
                )
                swaps += 1
                swapped = True

        if not swapped:
            break

    return numbers, comparisons, swaps

def selection_sort(values):
    numbers = values.copy()
    comparisons = 0
    swaps = 0
    for start in range(len(numbers) - 1):
        smallest = start
        for index in range(start + 1, len(numbers)):
            comparisons += 1
            if numbers[index] < numbers[smallest]:
                smallest = index

        if smallest != start:
            numbers[start], numbers[smallest] = (
```

```

        numbers[start], numbers[smallest] = (
            numbers[smallest], numbers[start]
        )
        swaps += 1

    return numbers, comparisons, swaps
def insertion_sort(values):
    numbers = values.copy()
    comparisons = 0
    shifts = 0

    for index in range(1, len(numbers)):
        current = numbers[index]
        position = index

        while position > 0:
            comparisons += 1

            if numbers[position - 1] <= current:
                break

            numbers[position] = numbers[position - 1]
            shifts += 1
            position -= 1

        numbers[position] = current
    return numbers, comparisons, shifts
results = {
    "Bubble Sort": bubble_sort(data),
    "Selection Sort": selection_sort(data),
    "Insertion Sort": insertion_sort(data),
}
print("Original data:", data)
print()
for algorithm, (sorted_data, comparisons, movements) in results.items():
    movement_name = "shifts" if algorithm == "Insertion Sort" else "swaps"
    print(algorithm)
    print("    Sorted data:", sorted_data)
    print("    Comparisons:", comparisons)
    print(f"    {movement_name.title()}: ", movements)
    print()

```

4.output

```

Original data: [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]

Bubble Sort
Sorted data: [5, 10, 12, 23, 34, 45, 56, 67, 78, 89]
Comparisons: 44
Swaps: 24

Selection Sort
Sorted data: [5, 10, 12, 23, 34, 45, 56, 67, 78, 89]
Comparisons: 45
Swaps: 6

Insertion Sort
Sorted data: [5, 10, 12, 23, 34, 45, 56, 67, 78, 89]
Comparisons: 31
Shifts: 24

```

5.Observation: All three algorithms successfully sort the same input list, but their comparisons and swaps/shifts differ.

Insertion Sort is generally more efficient for nearly sorted data because it requires fewer shifts.

The experimental results can be compared with the theoretical $O(n^2)$ time complexity of these algorithms.

#

4 question: – Optimising a Student Search and Sort Program

Prompt : "Analyze the student records [("Arun",82), ("Bala",65), ("Chitra",91), ("Divya",74), ("Esha",88)]. Create functions to find the highest mark, sort students by marks, and search by name. Identify unnecessary operations and suggest an efficient approach, then compare the time complexity of the original and improved solutions."

Code:

```
students = [
    ("Arun", 82),
    ("Bala", 65),
    ("Chitra", 91),
    ("Divya", 74),
    ("Esha", 88),
]

# Straightforward solution
def highest_mark_simple(students):
    highest_student = students[0]

    for student in students[1:]:
        if student[1] > highest_student[1]:
            highest_student = student

    return highest_student

def sort_students_simple(students):
    sorted_students = students.copy()

    for start in range(len(sorted_students) - 1):
        highest_position = start

        for position in range(start + 1, len(sorted_students)):
            if sorted_students[position][1] > sorted_students[highest_position][1]:
                highest_position = position

        sorted_students[start], sorted_students[highest_position] = (
            sorted_students[highest_position],
            sorted_students[start],
        )

    return sorted_students

def search_name_simple(student, name):
    for student in students:
        if student[0] == name:
            return student

    return None

# Improved solution
def highest_mark_improved(students):
    # max performs the same one pass operation with a clear key.
    return max(students, key=lambda student: student[1])

def sort_students_improved(students):
    # sorted is implemented and optimized in Python's standard library.
```

```

def sort_students_improved(students):
    # sorted is implemented and optimized in Python's standard library.
    return sorted(students, key=lambda student: student[1], reverse=True)
def create_name_index(students):
    return {(name, mark) for name, mark in students}
def search_name_improved(name_index, name):
    return name_index.get(name)
print("Students:", students)
print()
print("Straightforward solution")
print("Highest mark:", highest_mark_simple(students))
print("Sorted by mark:", sort_students_simple(students))
print("Search for Chitra:", search_name_simple(students, "Chitra"))
print("Search for Farah:", search_name_simple(students, "Farah"))
print()

name_index = create_name_index(students)

print("Improved solution")
print("Highest mark:", highest_mark_improved(students))
print("Sorted by mark:", sort_students_improved(students))
print("Search for Chitra:", search_name_improved(name_index, "Chitra"))
print("Search for Farah:", search_name_improved(name_index, "Farah"))
print()

```

output :

```

Students: [('Arun', 82), ('Bala', 65), ('Chitra', 91), ('Divya', 74), ('Esha', 88)]

Straightforward solution
Highest mark: ('Chitra', 91)
Sorted by mark: [('Chitra', 91), ('Esha', 88), ('Arun', 82), ('Divya', 74), ('Bala', 65)]
Search for Chitra: ('Chitra', 91)
Search for Farah: None

Improved solution
Highest mark: ('Chitra', 91)
Sorted by mark: [('Chitra', 91), ('Esha', 88), ('Arun', 82), ('Divya', 74), ('Bala', 65)]
Search for Chitra: ('Chitra', 91)
Search for Farah: None

```

Optimisation: avoid scanning the list again for every repeated name search; build a dictionary once, changing each later search from $O(n)$ to $O(1)$ average. Both versions sort in $O(n \log n)$, while the one-pass highest-mark scan is $O(n)$. The improved approach uses $O(n)$ extra space for the name index; the basic approach uses $O(1)$ extra search space and is fine for occasional lookups.

Observation : The program successfully finds the highest mark, sorts the students, and searches for a student by name. The improved approach can reduce unnecessary repeated operations by using suitable data structures and efficient searching. The original and improved solutions can be compared based on their time complexity and number of operations.

5 question: **Optimisation Challenge: Best Combination**

Prompt : "For the given items and budget of ₹50,000, find the combination of items with the maximum total price without exceeding the budget. Compare Brute Force, Greedy, and Dynamic Programming approaches, identify which method guarantees an optimal solution, and explain its time and space complexity. Test the solution with at least three different budgets."

Code:

```
from itertools import combinations
items = [
    ("Laptop", 60000),
    ("Phone", 30000),
    ("Tablet", 20000),
    ("Watch", 10000),
    ("Headphones", 5000),
]
budget = 50000

def brute_force(items, budget):
    """Try every possible subset and return the best valid subset."""
    best_items = []
    best_value = 0

    for size in range(len(items) + 1):
        for chosen_items in combinations(items, size):
            total_value = sum(value for _, value in chosen_items)

            if total_value <= budget and total_value > best_value:
                best_items = list(chosen_items)
                best_value = total_value

    return best_items, best_value

def greedy(items, budget):
    """Choose the most expensive remaining item first."""
    chosen_items = []
    total_value = 0

    for item in sorted(items, key=lambda item: item[1], reverse=True):
        value = item[1]

        if total_value + value <= budget:
            chosen_items.append(item)
            total_value += value

    return chosen_items, total_value

def dynamic_programming(items, budget):
    """Solve the 0/1 knapsack problem and reconstruct the chosen items."""
    item_count = len(items)
    table = [[0] * (budget + 1) for _ in range(item_count + 1)]

    for item_index in range(1, item_count + 1):
        value = items[item_index - 1][1]
```

```

        value = items[item_index - 1][1]

        for current_budget in range(budget + 1):
            table[item_index][current_budget] = table[item_index - 1][current_budget]

            if value <= current_budget:
                with_item = table[item_index - 1][current_budget - value] + value
                table[item_index][current_budget] = max(
                    table[item_index][current_budget], with_item
                )

        chosen_items = []
        current_budget = budget

        for item_index in range(item_count, 0, -1):
            if table[item_index][current_budget] != table[item_index - 1][current_budget]:
                item = items[item_index - 1]
                chosen_items.append(item)
                current_budget -= item[1]

        chosen_items.reverse()
        return chosen_items, table[item_count][budget]

def display_result(algorithm_name, result):
    chosen_items, total_value = result
    item_names = [name for name, _ in chosen_items]
    print(f"{algorithm_name}: {item_names}")
    print("    Total value:", total_value)

print("Items:", items)
print("Starting budget:", budget)
print()
print("Comparison for the starting budget")
display_result("Brute Force", brute_force(items, budget))
display_result("Greedy", greedy(items, budget))
display_result("Dynamic Programming", dynamic_programming(items, budget))
print()
print("Tests with different budgets")
for test_budget in [35000, 50000, 70000]:
    selected_items, total_value = dynamic_programming(items, test_budget)
    names = [name for name, _ in selected_items]
    print(f"Budget {test_budget}: {names}, total value = {total_value}")
print()

```

Output:

```

Items: [('Laptop', 60000), ('Phone', 30000), ('Tablet', 20000), ('Watch', 10000), ('Headphones', 5000)]
Starting budget: 50000

Comparison for the starting budget
Brute Force: ['Phone', 'Tablet']
    Total value: 50000
Greedy: ['Phone', 'Tablet']
    Total value: 50000
Dynamic Programming: ['Phone', 'Tablet']
    Total value: 50000

Tests with different budgets
Budget 35000: ['Phone', 'Headphones'], total value = 35000
Budget 50000: ['Phone', 'Tablet'], total value = 50000
Budget 70000: ['Laptop', 'Watch'], total value = 70000

```

Observation:

The goal is to maximize the total item value without exceeding the given budget. Brute Force and Dynamic Programming can guarantee an optimal solution, while Greedy does not always guarantee the best combination. Dynamic Programming is more efficient than Brute Force for larger inputs, but requires additional memory